PRACTICAL 12

CI/CD Pipeline Using Jenkins, Docker and Kubernetes

Automated build, containerization and deployment of an Iris AI application

Student Name	_____
Roll Number	_____
Class / Department	_____
Subject	_____
College	_____
Academic Year	_____

1. Aim

To set up a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Jenkins, Docker and Kubernetes to automate the validation, containerization and deployment of an Iris machine-learning application.

2. Objectives

• Connect a GitHub repository to Jenkins.

• Validate the Python application before packaging.

• Build Docker images for the Flask AI API and Nginx frontend.

• Load images into a local Minikube cluster.

• Deploy MongoDB, the AI application and frontend to Kubernetes.

• Verify deployments, pods and services.

3. Software and Tools Required

Tool	Purpose
Ubuntu on WSL2	Linux environment
Git and GitHub	Source-code version control
Jenkins	Pipeline orchestration and SCM polling
Docker	Build and package container images
Minikube	Local Kubernetes cluster
kubectl	Kubernetes command-line tool
Python 3	Python syntax validation
Flask, scikit-learn, PyMongo	AI API, prediction and database connectivity
Nginx	Frontend web server

4. Project Architecture

GitHub stores the source code. Jenkins checks out the repository, validates Python syntax, builds the AI and frontend images, loads them into Minikube and applies Kubernetes manifests. Kubernetes runs MongoDB, the Flask AI API and the Nginx frontend.

```
Developer -> git push -> GitHub (main)
                         |
                    Jenkins Pipeline
                         |
              +----------+----------+
              |          |          |
          Validate    Build Docker  Load images
          Python      AI + frontend into Minikube
                         |
                    kubectl apply
                         |
                 Kubernetes / Minikube
                   |        |        |
                Mongo    Iris AI  Frontend
```

5. Project Directory Structure

```
docker-compose-ai/
|-- ai-model/ (Dockerfile, app.py, requirements.txt)
|-- frontend/ (Dockerfile, index.html)
|-- kubernetes/
|   |-- ai-deployment.yaml
|   |-- ai-service.yaml
|   |-- frontend-deployment.yaml
|   |-- frontend-service.yaml
|   |-- mongodb.yaml
|-- docker-compose.yml
|-- Jenkinsfile
```

6. Procedure

Step 1: Prepare the GitHub repository

The local project was initialized as a Git repository, configured with the student's Git identity, connected to GitHub and pushed to the main branch.

```
cd ~/docker-compose-ai
git config --global user.name "Saish Malgundkar"
git config --global user.email "YOUR_GITHUB_EMAIL"
git remote set-url origin https://github.com/saish340/docker-compose-ai.git
git add .
git commit -m "Initial commit for AI CI CD pipeline"
git push -u origin main
```

Repository: https://github.com/saish340/docker-compose-ai.git

Step 2: Configure Jenkins access

Jenkins runs as the Linux user jenkins. Docker group access and Kubernetes configuration were set up for that service account.

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
sudo -u jenkins docker ps
sudo -u jenkins kubectl get nodes
sudo -u jenkins minikube status
```

The checks confirmed Jenkins could access Docker, kubectl and the running Minikube cluster.

Step 3: Add the Jenkinsfile

A Jenkinsfile was created at the root of the repository. BUILD_NUMBER provides a unique image tag for each pipeline build.

```
pipeline {
  agent any
  environment {
    AI_IMAGE = "iris-ai-model:ci-${BUILD_NUMBER}"
    FRONTEND_IMAGE = "iris-frontend:ci-${BUILD_NUMBER}"
  }
  stages {
    stage('Checkout') {
      steps { echo 'Source code checked out from GitHub' }
    }
    stage('Validate Python Application') {
      steps {
        sh 'python3 -m py_compile ai-model/app.py'
      }
    }
    stage('Build Docker Images') {
      steps {
        sh '''
          docker build -t "$AI_IMAGE" ./ai-model
          docker build -t "$FRONTEND_IMAGE" ./frontend
        '''
      }
    }
    stage('Load Images into Minikube') {
      steps {
        sh '''
          minikube image load "$AI_IMAGE"
          minikube image load "$FRONTEND_IMAGE"
        '''
      }
    }
    stage('Deploy MongoDB') {
      steps {
        sh '''
          kubectl apply -f kubernetes/mongodb.yaml
          kubectl rollout status deployment/mongodb --timeout=180s
        '''
      }
    }
    stage('Deploy AI Application') {
      steps {
        sh '''
          kubectl apply -f kubernetes/ai-deployment.yaml
```

```
          kubectl apply -f kubernetes/ai-service.yaml
          kubectl set image deployment/iris-ai-deployment iris-ai-container="$AI_IMAGE"
          kubectl rollout status deployment/iris-ai-deployment --timeout=180s
        '''
      }
    }
    stage('Deploy Frontend') {
      steps {
        sh '''
          kubectl apply -f kubernetes/frontend-deployment.yaml
          kubectl apply -f kubernetes/frontend-service.yaml
          kubectl set image deployment/iris-frontend-deployment iris-frontend-container="$FRONTEND_IMAGE"
          kubectl rollout status deployment/iris-frontend-deployment --timeout=180s
        '''
      }
    }
    stage('Verify Deployment') {
      steps {
        sh '''
          kubectl get pods
          kubectl get services
          kubectl get deployments
        '''
      }
    }
  }
  post {
    success { echo 'CI/CD Pipeline completed successfully!' }
    failure { echo 'CI/CD Pipeline failed. Check the Jenkins console output.' }
  }
}
```

Step 4: Configure the Jenkins job

• Open Jenkins at http://localhost:8080 and select New Item.

• Create a Pipeline job named Iris-AI-CI-CD.

• Select Pipeline script from SCM and choose Git.

• Repository URL: https://github.com/saish340/docker-compose-ai.git

• Branch specifier: */main; Script Path: Jenkinsfile.

• For a public repository, credentials can be left empty.

• Enable Poll SCM under Build Triggers.

SCM polling schedule:

```
H/5 * * * *
```

This checks the repository approximately every five minutes for changes. It does not trigger a build when there are no new changes.

Step 5: Run the pipeline

The Jenkins job Iris-AI-CI-CD was run using Build Now. Jenkins checked out commit 541784263edf213c4d45fb0977c526ae755ec11f from main.

• Validated Python syntax using python3 -m py_compile.

• Built iris-ai-model:ci-2 and iris-frontend:ci-2.

• Loaded both images into Minikube.

• Applied the MongoDB manifest and waited for its rollout.

• Applied the AI Deployment and Service, updated the image and waited for rollout.

• Applied the frontend Deployment and Service, updated the image and waited for rollout.

• Listed pods, services and deployments for verification.

7. Kubernetes Resources

Component	Resource	Purpose
MongoDB	mongodb	Database deployment and service
Storage	mongo-pvc	Persistent volume claim
AI API	iris-ai-deployment	Runs Flask prediction API
AI Service	iris-ai-service	Internal access on port 5000
Frontend	iris-frontend-deployment	Runs Nginx frontend
Frontend Service	iris-frontend-service	NodePort access on 30080

8. Verification Commands

```
kubectl get pods
kubectl get deployments
kubectl get services

# Open the frontend
minikube service iris-frontend-service

# Forward the AI API to localhost
kubectl port-forward service/iris-ai-service 5000:5000

# Test API from another terminal
curl http://localhost:5000/

# Test an Iris prediction
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features":[5.1,3.5,1.4,0.2]}'
```

9. Observed Output

The supplied Jenkins Console Output recorded the following:

• Python syntax validation successful.

• Both Docker images built successfully.

• Both images loaded into Minikube successfully.

• MongoDB deployment successfully rolled out.

• AI deployment successfully rolled out.

• Frontend deployment successfully rolled out.

• Kubernetes verification listed pods, services and deployments.

• Final Jenkins result: Finished: SUCCESS.

At verification time, MongoDB, AI and frontend each showed 1/1 available replicas. Old pods appeared in Terminating state during rolling updates while replacement pods were Running and Ready.

10. Result

The CI/CD pipeline was successfully configured using Jenkins, Docker and Kubernetes. Jenkins retrieved source code from GitHub, validated the Python application, built Docker images for the AI API and frontend, loaded the images into Minikube, deployed MongoDB and both application components, and verified the Kubernetes rollout. The Jenkins build completed with the status Finished: SUCCESS.

11. Conclusion

This practical demonstrated an automated development-to-deployment workflow. GitHub served as the source repository, Jenkins orchestrated the pipeline, Docker packaged the application components, and Kubernetes managed their deployment in Minikube. SCM polling was configured to detect repository changes and initiate subsequent builds.

12. Viva Questions and Answers

Q. What is CI/CD?
A. Continuous Integration and Continuous Deployment/Delivery automate code integration, validation, packaging and release.

Q. What is Jenkins?
A. An automation server used to orchestrate build, test and deployment pipelines.

Q. What is a Jenkinsfile?
A. A text file that defines a Jenkins Pipeline as code and is commonly stored with source code.

Q. What is Docker?
A. A platform for packaging applications and dependencies into container images.

Q. What is a Docker image?
A. A read-only template used to create containers.

Q. What is Kubernetes?
A. A container orchestration platform for deploying and managing containerized workloads.

Q. What is Minikube?
A. A tool for running a local Kubernetes cluster for learning and development.

Q. What is kubectl?
A. The command-line tool for communicating with a Kubernetes cluster.

Q. Why deploy MongoDB first?
A. The Flask application connects to MongoDB during startup and waits for the database to respond.

Q. What is a Deployment?
A. A Kubernetes resource that manages ReplicaSets and maintains the desired number of pods.

Q. What is a Service?
A. A stable network endpoint for accessing a group of pods.

Q. What is NodePort?
A. A Service type that exposes an application on a port of each cluster node.

Q. What is SCM polling?
A. A scheduled check of the source repository for changes that can trigger a pipeline build.

Q. Why use BUILD_NUMBER in image tags?
A. It gives each build a distinct image tag for identifying the version produced.

Q. What does rollout status do?
A. It waits for an updated Kubernetes Deployment to complete and its replicas to become available.

13. Screenshots to Attach

• Jenkins job dashboard showing successful build.

• Jenkins Console Output ending with Finished: SUCCESS.

• Jenkins pipeline stage view.

• Output of kubectl get pods.

• Output of kubectl get deployments and kubectl get services.

• Frontend page running through Minikube.

• Successful API home and prediction responses.

Attach actual screenshots from your system according to your college's practical-record format. This report records the console output supplied during the practical; it does not fabricate screenshots.